

Clew 코드 트랙 작업 기록

인제스트 필드-하드닝(2.5단계) · 리포트 & CLI(3단계)

수행한 전체 작업 · 현재 상태 · 향후 방향

작성일: 2026년 6월 12일 · Custos / Clew

목차

목차	2
1. 개요 — 이 채팅방에서 무슨 일이 있었나	3
2. 출발점 — 핸드오프가 전한 상황	5
3. 트랙 결정 — (1) 랩 마무리 먼저, 단 timebox	6
4. 2.5단계 — 인제스트 필드-하드닝 (상세)	6
4.1 진행 절차: 기준 먼저, 빌드는 나중	6
4.2 구현된 변환 3종	7
4.3 플랜 검토에서 잡은 결함 3건	7
4.4 빌드 중 이탈과 공식 진입점	7
4.5 결과	8
5. 3단계 — 리포트 & CLI (상세)	9
5.1 왜 이 작업인가	9
5.2 사전 등록과 범위	9
5.3 스테이지 가드 충돌과 처리 원칙	9
5.4 리포트 설계의 핵심 결정들	10
5.5 D5 검수에서 잡은 2건 — 언어와 면책	10
5.6 결과	10
6. 현재 상태	12
6.1 준비 완료된 것	12
6.2 정직 경계 (갱신)	12
7. 앞으로의 방향	13
7.1 이 방(코드 트랙)의 다음 트리거	13
7.2 아웃리치 트랙 (글 채팅방, 2주 timebox)	13
7.3 대기 모드의 자기기만 경계선	13
8. 이번 작업으로 확립·재확인된 원칙	15

이 채팅방은 부트스트랩 핸드오프(필드 테스트 결과 포함)를 이어받아 시작했고, 하루 동안 두 개의 빌드 단계를 완주했다: **2.5단계(인제스트 필드-하드닝)**와 **3단계(리포트 & CLI)**. 결과적으로 Clew는 “외부에서 트레이스 파일이 도착하면 몇 분 안에 정직한 영문 낭비 리포트를 돌려줄 수 있는” 상태가 됐다. 테스트는 155개에서 158개로 늘었고, 동결된 탐지기(detect/)와 평가 체계에는 단 한 줄도 손대지 않았다.

출발 결정 2.5단계 3단계 현재

● ● ● ● ●

핸드오프 흡수
(필드테스트 3발견)

트랙 (1) 선택
랩 timebox

인센티브
필드-하드닝
R1~R6 · 155 green

리포트 & CLI
D1~D5 · 158 green

트레이스 수신
대기 모드

단계	산출물	테스트	git 태그
출발	핸드오프 흡수 + 트랙 (1) 결정	155 (베이스라인)	—
2.5단계	라우터 필터 · collapse 승격 · JSON 추출 · 공식 진입점	155 (R1~R6 포함)	ingest-hardening-v1
3단계	save/load · capture · 리포트(md/json) · CLI · README	158 (D1~D3 포함)	report-cli-v1
현재	트레이스 수신 대기 모드	158 green	—

쉽게 말하면

비유하면, 지난 채팅들에서 '그물(탐지기)'을 완성하고 수조 시험까지 통과했다면, 이 채팅방에서는 두 가지를 했습니다. 첫째, 시험 항해에서 발견된 배(데이터를 그물까지 운반하는 장치)의 결함 3개를 수리했고(2.5단계), 둘째, 잡은 것을 보고서로 만들어 돌려주는 '포장·배송 시스템'(3단계)을 만들었습니다. 이제 남은 물고기(트레이스)만 보내주면 바로 결과를 돌려줄 수 있습니다.

2. 출발점 — 핸드오프가 전한 상황

이 채팅방은 이전 세션의 핸드오프 문서로 시작했다. 핵심 맥락: 합성 검증은 GO(F1 0.8571, 오탐 0)로 끝났고, 그 직후 실제 LangGraph 앱에 탐지기를 처음 붙여본 '필드 테스트'에서 세 가지 발견이 있었다.

- **발견 1 — llm span 뭉침:** 실제 계측에서 llm 호출 스펠들이 전부 모델 클래스명(FakeListLLM)으로 뭉쳐, 탐지기가 '같은 노드 3 회 반복'으로 오인해 거짓 경보를 냈다. 탐지기 잘못이 아니라 어댑터의 신원(identity) 결함. 임시 해결책 collapse(llm 스펠을 부모 노드로 접기)가 샌드박스에 있었다.
- **발견 2 — 라우터 거짓 양성:** LangGraph 의 분기 함수(should_loop 같은 제어흐름)도 스펠으로 계측되는데, 반복되면 노드처럼 중복 후보가 되어 거짓 경보. 구별 신호는 'llm/tool 자식이 있는가' — 토큰을 안 쓰는 스펠은 정의상 토큰 낭비가 아니므로 제외하는 게 원칙적으로 옳다.
- **발견 3 — 코사인 부풀림과 ϕ transfer 문제:** 출력이 JSON 형식으로 감싸져 있으면 그 포장(스캐폴드)만으로 코사인이 ~0.2 부풀어, 서로 다른 내용까지 임계값 $\phi=0.514$ 를 넘었다. 포장 제거는 수정 가능하지만, 제거 후에도 실제 출력의 유사도 분포(0.48~0.57)에 ϕ 가 한가운데 걸리는 더 큰 문제(ϕ 의 실측 transfer 미검증)가 남았다 — 이걸 실제 데이터로만 재보정 가능.

핸드오프는 방향 결정을 묻고 있었다: (1) 랩 마무리(어댑터 수정) 먼저 vs (3) 바로 실제 트레이스/수요 발굴.

쉽게 말하면

탐지기 자체는 멀쩡한데, 현실 세계의 데이터를 탐지기에 넣어주는 '입구'에 결함이 세 개 있었다는 이야기입니다. 안경(탐지기)은 좋은데 렌즈 앞에 김(어댑터 결함)이 서린 상태였고, 김 닦기를 먼저 할지, 김 서린 채로 일단 밖에 나갈지를 정해야 했습니다.

3. 트랙 결정 — (1) 랩 마무리 먼저, 단 timebox

결정: (1) 랩 마무리 먼저(하루 timebox), 끝나면 즉시 (3) 실제 트레이스/수요 발굴.

근거 두 가지. (a) 실제 트레이스를 받기 시작했는데 어댑터가 이미 알려진 결함으로 거짓 경보를 내면 첫인상에서 신뢰를 잃는다 — 진단이 끝난 확정 결함이라 수리가 싸고 빠르다. (b) 반대로 랩에 계속 머무는 것은 '수요 질문 회피'라는 자기기만이 될 수 있어 timebox로 묶었다. ϕ -transfer 측정(선택지 2)은 지금 합성 환경에서 또 돌리는 것보다, 실제 트레이스가 들어오면 그 데이터로 하는 게 더 정직하다고 보고 뒤로 미뤘다.

쉽게 말하면

'김부터 닦되, 닦는 데 하루만 쓴다'고 정한 것입니다. 손님(트레이스 제공자)을 맞기 전에 알려진 흠은 고치는 게 예의이고, 동시에 청소가 외출 회피의 핑계가 되지 않게 시간 제한을 걸었습니다.

4. 2.5 단계 — 인제스트 필드-하드닝 (상세)

4.1 진행 절차: 기준 먼저, 빌드는 나중

작업은 3분할 work-kit으로 진행됐다: (A) SPEC §9에 변환 3종과 합격 기준 R1~R4를 사전 등록 → (B) 그 기준을 git에 커밋하고 브랜치 생성, 베이스라인 테스트 확인 → (C) plan 모드로 구현 계획을 받아 검토·승인 후 빌드. 결과를 보기 전에 합격선을 못 박는 순서는 이 프로젝트의 모든 단계와 동일하다.

4.2 구현된 변환 3 종

변환	내용	핵심 원칙
라우터 제외	llm/tool 자손이 없는 chain 스패(제어흐름)을 탐지 단위에서 제외	토큰 0 스패는 정의상 토큰 낭비가 아니다 (hack이 아닌 원칙)
collapse 승격	llm 서브스팬을 부모 노드로 접기 + 접기 전 토큰/비용을 부모로 합산 + ReAct 구조의 tool 자식은 re-parent	비용 스토리 보존, tool은 절대 접지 않음(requery 게이트 대상)
JSON 내용 추출	output.value JSON에서 내용만 추출해 깨끗한 output_text 생성	포장(스캐폴드)이 아닌 내용을 임베딩한다

4.3 플랜 검토에서 잡은 결함 3 건

Claude Code의 첫 플랜은 골격이 좋았지만, 승인 전 검토에서 세 가지를 고치게 했다:

- **고아 스패 처리 누락:** llm 스패를 제거할 때 그 자식(특히 tool)의 부모가 끊겨 댕글링되는 문제. 제거되는 llm 의 부모로 re-parent 하도록 수정하고 R5 단위 테스트를 추가시켰다.
- **판정 기준이 '직계 자식'뿐:** llm 호출이 손자 깊이에 있는 진짜 작업 노드가 라우터로 오인되어 제거될 수 있었다. '자손(descendant)' 기준으로 변경.
- **추출 규칙이 '첫 번째 문자열':** 키 순서에 의존해 status 같은 엉뚱한 필드를 집을 수 있었다. '재귀 순회 후 가장 긴 문자열 leaf'로 변경(결정론적).

4.4 빌드 중 이탈과 공식 진입점

빌드 중 Claude Code가 플랜에서 벗어난 지점이 있었다: 어댑터에 전처리를 내장하려다 기존 어댑터 테스트(raw 변환 가정)와 충돌하자, 전처리를 호출자가 명시적으로 불러야 하는 구조로 후퇴했다. 기존 테스트를 안 건드린 판단은 옳았지만, 이대로면 누군가 전처리 호출 한 줄을 까먹는 순간 방금 고친 결함 3개가 조용히 전부 되살아난다 — 테스트는 green인데 시스템은 틀린, 가장 경계하는 형태. 그래서 **공식 진입점 ingest_otel_spans()**(= raw 변환 + 전처리)를 추가시키고, 그 진입점이 전처리를 실제로 적용함을 핀 고정하는 R6 테스트를 붙였다. 또한 R4가 추

출 함수를 문자열에 직접 호출해 통과한 것이라 '파이프라인이 그 함수를 실제로 거치는지'는 아무도 안 보고 있었는데, 이것도 R6가 함께 검증하게 했다.

4.5 결과

- 회귀 테스트 R1~R6 신설: R1 깨끗 트레이스 FP=0 / R2 repeat_node TP 유지 / R3 라우터 적대 케이스 FP 소거 / R4 JSON 추출 후 코사인 하락 / R5 ReAct re-parent / R6 공식 진입점 핀 고정.
- 155 개 테스트 전부 green, 누수 가드 11/11, detect/ 변경 0.
- 커밋-태그: ingest-hardening-v1, master 머지 완료. 랩 timebox 종료.

쉽게 말하면

이 단계의 백미는 4.4입니다. '고치긴 했는데 아무도 그 수리된 길로 다니라고 강제하지 않는' 상태를 발견하고, 공식 출입구를 하나로 정한 뒤 그 출입구가 진짜로 수리된 길을 지나는지 감시 카메라(R6 테스트)까지 단 것입니다. 점수(테스트 통과)만 보지 않고 '그 점수가 실제 사용 경로를 보증하는가'를 따진 사례입니다.

5. 3 단계 — 리포트 & CLI (상세)

5.1 왜 이 작업인가

아웃리치가 약속하는 것은 “트레이스를 주면 낭비 리포트로 돌려준다”인데, 그 리포트를 만드는 코드가 없었다. SPEC 로드맵의 report 단계가 빈칸이었고, CLI도 마크다운 리포트도 없었다. 게다가 “데이터가 못 나오면 코드를 보내줄 테니 로컬에서 돌려라”라는 프라이버시 옵션은 남아 설치해서 돌릴 수 있는 패키징을 전제한다. DM에 트레이스가 도착했을 때 몇 시간 안에 터어라운드하려면 지금 만들어야 했다.

5.2 사전 등록과 범위

SPEC §10에 합격 기준 D1~D5를 결과 보기 전에 등록하고 커밋했다. 범위: (1) 트레이스 JSON 직렬화 왕복(save_trace/load_trace) + 캡처 헬퍼(clew.capture), (2) report/ — 사람용 markdown + 기계용 JSON, (3) CLI ``python -m clew analyze``, (4) README quickstart 3단계. ϕ /N/모델은 불변이며 CLI에 임계값 오버라이드 옵션을 넣는 것 자체를 금지했다 — 실험은 field_test/ 샌드박스에서만.

5.3 스테이지 가드 충돌과 처리 원칙

구현 탐색 중 충돌이 발견됐다: 기존 테스트에 “3단계 전까지 report/는 비어 있어야 한다”는 가드가 있어, report/에 코드를 넣으면 무조건 깨진다. 여기서 중요한 원칙을 세웠다: **만료된 스테이지 경계 가드의 갱신은 허용, 동결된 검증(누수 가드·eval·CRITERIA)의 수정은 불가**. 우리는 3 단계를 사전 등록하고 진입했으므로 그 가드의 수명이 정상적으로 만료된 것이다. 단 삭제가 아니라 교체(허용 모듈 목록과 정확히 일치해야 통과하는 새 가드)로, 가드의 감시 기능은 유지하면서 경계만 옮겼고, 사유와 커밋 해시를 docstring과 SPEC에 남겼다. 빌드 중 같은 성격의 가드가 한 개 더(test_dod.py) 발견되어 동일 원칙으로 처리했고, 기록 정합을 위해 SPEC 주석에 2건 모두 명시하고 오해 소지가 있는 테스트 이름도 개명했다. 죽은 가드를 피하려고 코드 구조를 비트는 대안(2번)은 ‘테스트를 통과시키려 시스템을 왜곡하는’ 형태라 반려했다.

5.4 리포트 설계의 핵심 결정들

- **비용 귀속 규칙:** 낭비 토큰은 재등장(candidate) 스펠만 계산한다 — 원본(origin)은 정당한 1 회. 원본까지 합치면 낭비액이 과대계상되는데, 이는 우리가 가장 피해야 할 과장이다.
- **중복 제거(dedupe):** 한 candidate 가 여러 origin 과 쌍이 되면 표의 행이 중복돼 낭비가 부풀어 보인다. candidate 별 최고 코사인 쌍 1 개만 유지하고, 행 수 = 낭비 스펠 수임을 테스트로 단언.
- **프라이버시:** 출력 스니펫은 기본 80 자 절단 + --no-snippets 옵션(필드 자체 제외). markdown 과 JSON 양쪽에 동일 적용 — JSON 이 오히려 그대로 복붙되어 돌아올 가능성이 높아서, JSON 쪽 절단을 테스트로 단언했다.
- **token_count 가 없으면 “unknown”:** 숫자를 지어내지 않고 모른다고 표기한다.
- **리포트 머리에 동결 파라미터 인쇄:** $\phi=0.514345$, $N=2$, 모델명 — 어떤 설정으로 분석했는지 리포트가 스스로 말한다.

5.5 D5 검수에서 잡은 2 건 — 언어와 면책

D5 수동 확인(waste/clean 트레이스를 CLI로 실제 실행)에서 리포트 실물을 검수한 결과 두 가지를 고쳤다: (1) 리포트가 한국어였다 — 주 수신자는 해외 개발자이므로 영어를 기본으로 전환. (2) 면책 푸터가 없었다 — ϕ -transfer 미검증이 알려진 리스크이므로, 리포트 스스로 “임계값은 합성 트레이스로 보정됐고 실측 보정은 진행 중이며, 코사인 0.51 근처의 경계 매치는 사람의 검토가 필요하다”고 말하게 했다(markdown 푸터 + JSON note 필드). 오탐이 났을 때 신뢰를 지키는 보험이자 정직성 규칙의 연장이다.

5.6 결과

- 158 개 테스트 green(D1~D3 신설), 누수 가드 11/11, detect/ 변경 0.
- D5 실물 확인: waste 트레이스 → 영문 리포트(낭비 스펠 1, cosine 0.9274, unknown 정직 표기, 푸터 포함) / clean 트레이스 → “no waste detected”. 둘 다 종료코드 0.
- 커밋-태그: report-cli-v1, master 머지.

쉽게 말하면

이 단계는 '진단서 양식 만들기'였습니다. 진단(탐지)은 이미 할 수 있었지만, 환자에게 줄 문서가 없었던 것이죠. 만들면서 지킨 원칙이 셋입니다: 병을 부풀리지 않는다(재등장만 비용 계산, 중복 행 제거), 환자의 사생활을 지킨다(스니펫 절단), 그리고 진단서에 '이 검사법의 한계'를 스스로 적는다(면책 푸터). 모르는 수치는 모른다고 쓰고요.

6. 현재 상태

6.1 준비 완료된 것

- 외부인이 pip install → capture → analyze 3 단계로 로컬 실행 가능(README quickstart). 트래이스 파일이 도착하면 몇 분 안에 정직한 영문 리포트(추정 표기·unknown 처리·합성 보정 면책 푸터 포함) 반환 가능.
- git 태그 체인: stage1-freeze → stage2-detector-freeze → stage2-eval-go → ingest-hardening-v1 → report-cli-v1.
- 테스트 158 개 green, 누수 가드 11/11, 동결 탐지기 불변($\phi=0.514345$, $N=2$, paraphrase-multilingual-MiniLM-L12-v2).

6.2 정직 경계 (갱신)

말할 수 있는 것	여전히 금지
합성 검증 통과 (사전등록 단발 평가 GO: F1 0.8571, FPR 0)	"실측에서 작동한다" (ϕ transfer 미검증)
실계측 경로의 구조적 기제를 회귀 테스트로 고정 (R1~R6)	"실제 절감을 달성했다" (실측 0건)
리포트 턴어라운드 도구 완비 (D1~D5)	"프로덕션에서 검증됐다" (사용자 0명)

쉽게 말하면

'실측에서 작동한다'는 문장은 아직 쓸 수 없습니다. 그 문장은 코드를 더 쓴다고 생기는 게 아니라, 실제 트래이스가 와야만 생깁니다. 지금 상태는 그 트래이스가 오는 즉시 받아칠 수 있는 '완전 대기'입니다.

7. 앞으로의 방향

7.1 이 방(코드 트랙)의 다음 트리거

글 채팅방의 아웃리치에서 첫 실제 트레이스가 들어오는 순간이 이 방의 다음 작업 시작점이다. 그때의 순서:

- ① **분석 + 발송 전 검토**: 받은 파일을 analyze 로 돌리고, 결과를 보내기 전에 함께 검토한다. 특히 코사인 0.5 대의 경계 매치는 잠재 거짓 양성으로 의심하고 본다.
- ② **FP/TP 판정 기록**: 각 매치가 진짜 낭비인지 아닌지의 판정 근거를 기록한다 — 이것이 ϕ 실분포 재보정의 원자료가 된다.
- ③ **ϕ -transfer 실험 설계**: 트레이스 3~5 건이 모이면, 실측 분포 기반 ϕ 재보정 실험을 사전등록부터 설계한다. ϕ 를 개별 케이스에 맞춰 손대는 것은 그때도 금지 — v1 의 사후 오버핏과 같은 형태다.

7.2 아웃리치 트랙 (글 채팅방, 2 주 timebox)

- 사전 등록된 성공/중단 기준: 성공 = 실제 트레이스 3~5 건 + 통증 대화 5 건 이상 / 재검토 = 2 주간 20+ 접촉에 트레이스 0 건.
- 채널 순서: DEV.to 앵커 글(1 일차) → 한국 커뮤니티 동시(Instruct.KR·모두의연구소) → GitHub 콜드 DM 10~20 명(2~5 일차) → Reddit 은 2 주간 댓글만(카르마 적립) → HN 은 실측 케이스 1 건 생긴 후.
- 모든 글의 CTA 동일: "실제 트레이스 3~5 건을 찾는다 → 낭비 리포트로 돌려준다 → 데이터가 못 나오면 코드를 보내줄 테니 로컬에서 돌리고 숫자만 공유."

7.3 대기 모드의 자기기만 경계선

트레이스가 오기 전까지 이 방은 대기 모드다. 그동안 할 일은 Reddit 댓글 카르마 적립(글 방 담당)과 트레이스 기다리기 — **랩으로 도망치지 않기**. 탐지기를 더 다듬고 싶은 충동은 '수요 질문 회피'의 가장 흔한 형태이며, 지금 단계에서 가장 경계해야 할 자기기만이다. 다음 코드 작업

은 반드시 실제 데이터가 트리거한다.

쉽게 말하면

앞으로의 그림은 단순합니다. 글(아웃리치)이 물고기를 데려오고, 이 방(코드)은 도착 즉시 요리해서 돌려줍니다. 그 요리 결과 하나하나가 '진짜 바다에서도 그물이 작동하는가'라는 마지막 질문의 데이터가 됩니다. 그 전까지 주방을 더 꾸미는 일(랩 작업)은 하지 않기로 했습니다 — 그건 일하는 척이지 전진이 아니니까요.

8. 이번 작업으로 확립·재확인된 원칙

- **기준 먼저, 빌드 나중:** 두 단계 모두 합격 기준(R1~R6, D1~D5)을 결과 보기 전에 SPEC 에 등록하고 git 커밋한 뒤에야 구현에 들어갔다.
- **스테이지 가드 vs 동결 검증의 구분:** 수명이 만료된 단계 경계 가드는 사유·커밋 해시와 함께 '교체'할 수 있다(약화 아님). 누수 가드·eval-CRITERIA 같은 동결 검증은 불가침.
- **테스트 통과 ≠ 시스템 정상:** '수리는 됐지만 아무도 그 길로 안 다니는' 상태(전처리 미연결)를 공식 진입점 + 핀 고정 테스트로 막았다. 함수 단위 테스트만 있고 파이프라인 적용 검증이 없는 빈틈(R4 vs R6)도 같은 종류다.
- **리포트도 정직해야 한다:** 과대계상 방지(candidate 만 비용 계산, dedupe), unknown 정직 표기, 합성 보정 면책 푸터 — 산출물 자체가 과장 경계를 지키게 설계.
- **프라이버시는 기본값:** 스니펫 절단이 기본이고 완전 제외 옵션이 있다. 남의 데이터를 다루는 도구의 기본 자세.
- **동결물 불가침:** 두 단계 내내 detect/ 변경 0, ϕ /N/모델 불변. CLI 에 임계 오버라이드 옵션을 넣는 것조차 금지해 '편의'가 '사후 튜닝'의 뒷문이 되는 것을 차단.

현재 정확한 한 문장: "합성으로 검증된 탐지기 앞뒤로, 실계측 입구(인제스트)의 알려진 결함을 회귀로 고정해 수리했고, 출구(리포트·CLI)를 완비했다. 실측 검증은 첫 트레이스가 도착하는 순간 시작된다."